

# Physics and calculation methods

ZinCin Feb  
ruary 2026

## 1 Problem Statement

In the two-dimensional domain  $\Omega = [0, W] \times [0, H]$ , find the potential of each point  $\varphi_{x,y}$ . To recreate a potential similar to the Earth's, set the following boundary conditions:

1. Upper boundary ( $y = H$ ): fixed potential 1.0
2. Lower boundary ( $y = 0$ ): fixed potential 0.0
- 3-4. Side boundaries ( $x = 0$  and  $x = W$ ): Neumann condition  $\frac{\delta\varphi}{\delta n} = 0$   
 $\Rightarrow \frac{\delta\varphi}{\delta x} = 0$  — —

## 2 Laplace's Equation

In electrostatics, the potential  $\varphi$  in regions free of charge satisfies Laplace's equation:

$$\nabla^2 \varphi = \frac{\delta^2 \varphi}{\delta x^2} + \frac{\delta^2 \varphi}{\delta y^2} = 0$$

We introduce a uniform grid with a step of  $h = 1$  between the nodes. The grid nodes have coordinates  $0, 1, 2, \dots, W - 1, W$  and  $0, 1, 2, \dots, H - 1, H$

$$\left. \frac{\partial^2 \varphi}{\partial x^2} \right|_{i,j} \approx \varphi_{i,j-1} - 2\varphi_{i,j} + \varphi_{i,j+1},$$
$$\left. \frac{\partial^2 \varphi}{\partial y^2} \right|_{i,j} \approx \varphi_{i-1,j} - 2\varphi_{i,j} + \varphi_{i+1,j},$$

We substitute into Laplace's equation:

$$(\varphi_{i-1,j} - 2\varphi_{i,j} + \varphi_{i+1,j}) + (\varphi_{i,j-1} - 2\varphi_{i,j} + \varphi_{i,j+1}) = 0,$$

obtaining a five-point template:

$$\varphi_{i-1,j} + \varphi_{i+1,j} + \varphi_{i,j-1} + \varphi_{i,j+1} - 4\varphi_{i,j} = 0,$$

or the same thing:

$$\varphi_{i,j} = \frac{1}{4}(\varphi_{i-1,j} + \varphi_{i+1,j} + \varphi_{i,j-1} + \varphi_{i,j+1})$$

From this it follows that at each interior point the value of the potential must equal the arithmetic mean of the four neighbors.

### 3 Iterative Methods of Solution

Unlike the Optozorax solution, I did not use the Finite Element Method (FEM), which helps discretize space and creates a numerical method for solving differential equations, but used other iterative methods. Next, methods for calculating the potential throughout the entire simulation will be considered, and the superscripts  $k$  and  $k+1$  will denote the potential in the current and new frames, respectively.

#### 3.1 Jacobi Method

$$\varphi_{i,j}^{k+1} = \frac{1}{4}(\varphi_{i-1,j}^k + \varphi_{i+1,j}^k + \varphi_{i,j-1}^k + \varphi_{i,j+1}^k),$$

This is an acceptable method that I used before; however, now I have found more effective methods described below.

#### 3.2 Gauss-Seidel Method

At each iteration, we go through all the internal nodes and update the value according to the formula:

$$\varphi_{i,j}^{k+1} = \frac{1}{4}(\varphi_{i-1,j}^{k+1} + \varphi_{i+1,j}^k + \varphi_{i,j-1}^{k+1} + \varphi_{i,j+1}^k),$$

where the superscript  $k$  denotes the iteration. At the same time, the already computed values in the current iteration from the left and above are used (if the traversal goes row by row from top to bottom, left to right), which allows us to use the Red-Black SOR method.

#### 3.3 Successive Over-Relaxation Method SOR

SOR accelerates convergence by introducing a relaxation parameter  $\omega$ , satisfying the inequality  $1 < \omega < 2$ . First, the Gauss-Seidel value  $\tilde{\varphi}$  is calculated, and then the new value as a weighted sum of the old potential  $\varphi$  and the new  $\tilde{\varphi}$ :

$$\tilde{\varphi}_{i,j} = \frac{1}{4}(\varphi_{i-1,j}^{k+1} + \varphi_{i+1,j}^k + \varphi_{i,j-1}^{k+1} + \varphi_{i,j+1}^k),$$

$$\varphi_{i,j}^{k+1} = \varphi_{i,j}^k(1 - \omega) + \tilde{\varphi}_{i,j}\omega,$$

or an equivalent form:

$$\varphi_{i,j}^{k+1} = \varphi_{i,j}^k + \omega(\tilde{\varphi}_{i,j} - \varphi_{i,j}^k),$$

For  $\omega = 1$ , the method exactly coincides with Gauss-Seidel (since  $(1 - \omega) = (1 - 1) = 0$ ), and for  $1 < \omega < 2$  the potential is redirected towards the average, which accelerates the convergence of problems with a predominance of low-frequency error components (up to the accuracy of the error).

### 3.4 Red-Black SOR

In my project, I use Red-Black SOR. To avoid dependence on the traversal order and to ensure vectorization, I apply chessboard ordering. Let's divide all internal nodes into two sets: red (sum of indices is even) and black (sum of indices is odd). It is important that all neighbors of red nodes are black, and vice versa.

The iteration proceeds in two half-steps:

1. All red nodes are updated using the old values of the black neighbors.
2. All black nodes are updated using the newly computed values of the red neighbors.

Formula for the red half-wave ( $k \Rightarrow k + \frac{1}{2}$ ):

$$\tilde{\varphi}_{i,j}^{red} = \frac{1}{4}(\varphi_{i-1,j}^k + \varphi_{i+1,j}^k + \varphi_{i,j-1}^k + \varphi_{i,j+1}^k), (i,j) - Red$$

$$\varphi_{i,j}^{k+\frac{1}{2}} = \varphi_{i,j}^k(1 - \omega) + \tilde{\varphi}_{i,j}^{red}\omega$$

Formula for black half-step ( $k + \frac{1}{2} \Rightarrow k + 1$ ):

$$\tilde{\varphi}_{i,j}^{black} = \frac{1}{4}(\varphi_{i-1,j}^k + \varphi_{i+1,j}^k + \varphi_{i,j-1}^k + \varphi_{i,j+1}^k), (i,j) - Black$$

$$\varphi_{i,j}^{k+\frac{1}{2}} = \varphi_{i,j}^k(1 - \omega) + \tilde{\varphi}_{i,j}^{black}\omega$$

After that, we obtain an approximation, by repeating which many times one can achieve a quality approximation